

程序设计90

张宏斌 副教授

复习

- Python的程序结构
- 变量名的命名规则
- Python的数字类型
- 算术运算符: + - * / // % **
- 布尔值
- 逻辑运算符: and or not
- 比较运算符: > < >= <= == !=
- Python的常用内置函数
- print()和input()的用法
- 模块的导入方法
- math库中的常用数学函数

第二章（续） 字符串

- 1 掌握字符串的概念和用法
- 2 熟练运用字符串常用方法
- 3 熟练运用运算符和内置函数对字符串的操作



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



编程（解决问题的过程）

- 充分理解问题，精确确定目标
- 用计算思维分析问题
- 写出所有的输入、输出；写出逻辑过程
 - 如果不能手动写出过程，就不能自动化该过程
- 确定要实现目标，我们需要什么样的数据、输入、或原材料
- 确定实现期望目标的每一步过程（程序！程序！）
 - 即所谓的算法，也称伪代码
 - 指定纸的厚度
 - 指定地球到月亮的距离distance
 - 求最小整数n，使得 $2 ** n \geq distance$



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



编程（解决问题的过程）

- 将上一步的过程写成代码（当然Python）
 - 利用Python的所有元素——数据类型、函数、控制结构、类、.....
 - 检查是否有问题（debug）
- 最后，检查确定是否实现了目标
 - 和最终目标进行比较
 - 和第二步的输入输出比较

练习

- 奥密克戎病毒的传染指数 R_0 平均为3.77，即一位患者平均会传染给3.77个人
- 假设传播速度为1天，即1人1天平均传染3.77人
- 假设最初只有1人感染，请计算：
 - 按照上面的传染指数，10天后会有多少人染病？
 - 感染4亿人需要多少天？
- 假设最初有100人感染，请计算：
 - 按照上面的传染指数，3天后会有多少人染病？
 - 感染4亿人需要多少天？
- 提示：用`math.ceil()`向上取整

用计算思维分析

- 分解：变量定义（输入）、计算、输出
- 模式识别：建模
 - # 若最初1人感染，那么1天后感染人数是 $(1 + 3.77)$ ，取整
 - # n 天后，感染人数为 $(1 + 3.77)^n$ ，取整
 - # 感染4亿人需要 $\log(400000000, 1+3.77)$ 天，取整
- 抽象：变量、计算公式、输出
- 算法设计：

```
import math
```

```
# 定义传染指数
```

```
R0 = 3.77
```

```
patient_num = (1 + 3.77)**10
```

```
days = math.log(400000000, 4.77)
```

```
print("若最初1人感染，10天后会感染{:d}人".format(round(patient_num)))
```

```
print("感染4亿人，需要{:d}天".format(math.ceil(days)))
```

若最初100人感染，那么1天后感染人数是 $(100 + 377)$

n天后，感染人数为 $100 \times (1 + 3.77)^n$ ，取整

感染4亿人需要 $\log(4000000, 1+3.77)$ 天，取整

```
patient_num = 100 * (1 + 3.77)**3
```

```
days = math.log(4000000, 4.77)
```

```
print("若最初100人感染，3天后会感染{:,d}人".format(round(patient_num)))
```

```
print("感染4亿人，需要{:d}天".format(math.ceil(days)))
```



字符串简介

- 在Python中，字符串被用来表示文本数据
- 字符串属于不可变有序序列，使用单引号、双引号、三单引号或三双引号作为定界符，并且不同的定界符之间可以互相嵌套

'abc'、'123'、'中国'

"Python"

"""Tom said, "Let's go""""

- 一般用三重引号定义多行字符串：

- Long_str = ''' first line,
second line,
last line. '''

- 在Python中，不在语句中的三重引号，被自动被识别为注释
- 用函数str()可以将其他类型数据转换为字符串，如str(12.35)



字符串编码

- 计算机自己能理解的"语言"是二进制数；而人类所能理解的语言文字则是一套由英文字母、汉语汉字、标点符号字符、阿拉伯数字等等很多的字符构成的字符集。如果要让计算机来按照人类的意愿进行工作，则必须把人类所使用的这些字符集转换为计算机所能理解的二进制码，这个过程就是编码，他的逆过程称为解码
- Python 3.x默认使用UTF-8编码格式，完全支持中文字符，无论是一个数字、英文字母，还是一个汉字，在统计字符串长度时都按一个字符对待和处理

```
>>> s = '中国广东广州'
>>> len(s)          # 字符串长度，或者包含的字符个数
6
>>> 姓名 = '张三'    # 使用中文作为变量名
>>> print(姓名)      # 输出变量的值
张三
```

常用中文编码

- GBK
- GB2312
- utf-8
- utf-8-sig

确定Geany文档的默认编码规则

- 在 **编辑-首选项（设置）-文档** 中，设置默认编码规则为**utf-8**

转义字符

- 在 Python 中，通过**反斜杠 **来引入特殊的字符编码，例如使用 `\'` 引入单引号，或者不容易通过键盘输入的字符，例如 `\n` 表示一个换行符，`\t` 表示一个制表符（通常由 4 个空格组成）：

Python 常用转义字符

转义字符	意义
<code>\\</code>	反斜杠
<code>\'</code>	单引号
<code>\"</code>	双引号
<code>\n</code>	换行符
<code>\t</code>	水平制表符
<code>\r</code>	回车
<code>\xhh</code>	十六进制值 hh 对应的字符，例如 <code>\x61</code> 表示字符 a
<code>\uhhhh</code>	4 位十六进制值对应的 Unicode 字符，例如 <code>\u0061</code> 表示字符 a
<code>\ooo</code>	八进制值 ooo 对应的字符，例如 <code>\141</code> 表示字符 a

转义字符 (续)

- 转义字符用法:

```
>>> print('Hello\nWorld')          # 包含转义字符的字符串
```

```
Hello  
World
```

```
>>> print('\101')                  # 三位八进制数对应的字符
```

```
A
```

- 为了避免对字符串中的转义字符进行转义，可以使用原始字符串，在字符串前面加上字母**r**或**R**表示**原始字符串**，其中的**所有字符都表示原始的含义而不会进行任何转义**

```
>>> path = 'C:\Windows\notepad.exe'
```

```
>>> print(path)                    # 字符\n被转义为换行符
```

```
C:\Windows
```

```
otepad.exe
```

```
>>> path = r'C:\Windows\notepad.exe' # 原始字符串，任何字符都不转义
```

```
>>> print(path)
```

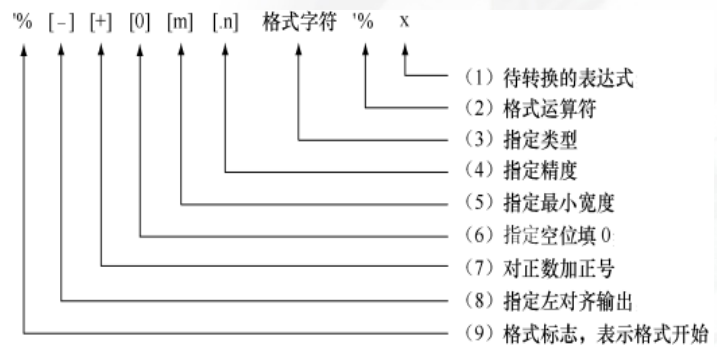
```
C:\Windows\notepad.exe
```

字符串格式化

- 使用**%**运算符进行格式化
- 使用**format()**方法进行格式化
- 格式化的字符串常量 (**f表达式**)
- 使用字符串方法格式化: **center()**, **just()**, **ljust()**

使用%运算符进行格式化

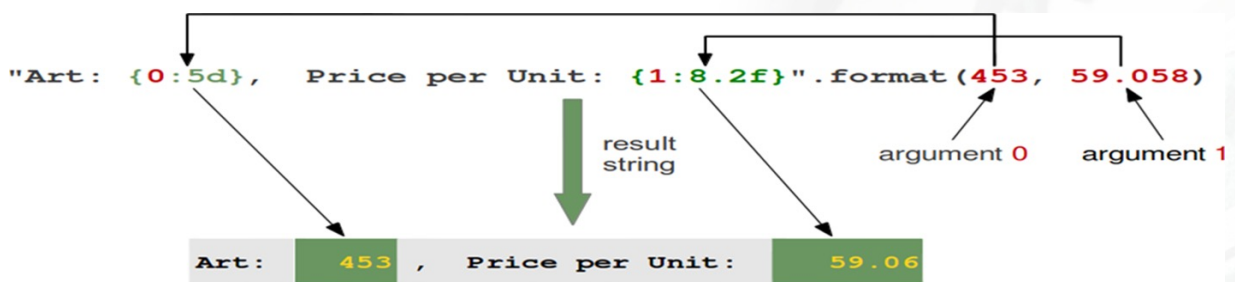
- '%-012.3f'%123.45



格式	说明	格式	说明
"%d"	格式化整数	"%s"	字符串
"%5d"	宽度5, 前面空格	"%9s"	字符串, 9位
"%05d"	宽度5, 前面0	"%-9s"	字符串, 左对齐
"%f"	浮点数	"%%"	%
"%.2f"	浮点数, 2位小数	"%o"	八进制
"%7.2f"		"%x"	十六进制
"%c"	类似于函数chr()	"%e"	指数记法

使用format()方法进行格式化

- 用{}代替%, 用方法format()引出输出的变量



- 如: `"{0:<20s} {1:6.2f}".format('Spam & Eggs:', 6.99)`
 - "<", ">", "^" 分别表示左、右对齐、居中
 - "," 表示千位数逗号隔开表示
- `x = 5897653423.89676, print("The value is {0:,.2f}".format(x))`
- 若{}中有格式说明符, 则其中的':'必不可少!

格式表示规则

- **format_spec ::=**
[[fill]align][sign][#][width][grouping_option][.precision][type]
 - **fill** ::= <any character>
 - **align** ::= "<" | ">" | "=" | "^"
 - **sign** ::= "+" | "-" | ""
 - **width** ::= digit+
 - **grouping_option** ::= "_" | ","
 - **precision** ::= digit+
 - **type** ::= "b" | "c" | "d" | "e" | "E" | "f" | "F" | "g" | "G" | "n" | "o" | "s" | "x" | "X" | "%"
- **#**用于整数，表示在二进制、八进制、十六进制数字前面应显示'0b'、'0o'、'0x'、'0X'；用于浮点数或复数，表示显示小数点（即使没有小数位）
- **注意：**精度用于字符串，则指输出字符的个数（后面第3例）



type参数		含义	例
默认		十进制整数	123
f		浮点数	123.123
%		百分比格式	12312.3%
e		指数形式	1.2e01
进制转换	d	十进制整数	123
		十进制	123
	b	二进制	1101111
	o	八进制	157
	x	十六进制	6f
	#x	小写表示的十六进制	0x6f
	#X	大写表示的十六进制	0X6F
c		字符，打印前转换为Unicode代码	'123'



format()方法举例

```
>>> print('{0:^8}{1:^10}{2:*<5}'.format('代码', '名称', '价格'))
+++代码+++....名称....价格***

>>> print('{:8}{:10}{:<5.2f}'.format('601988', '中国银行', 3.19))
601988   中国银行  3.19_

>>> print('{:+8.2}{:10}{:<5.1%}'.format('601288', '农业银行', 3.16))
+++60+++  农业银行  316.0%

>>> print('{:8}{:10}{:>10,d}'.format('601398', '工商银行', 494000))
601398   工商银行   494,000
```



format()方法举例

```
>>> print('{0:.3f}'.format(1/3))    # 保留3位小数
0.333

>>> '{0:%}'.format(3.5)             # 格式化为百分数
'350.000000%'

>>> '{0:_},{0:_x}'.format(1000000)   # Python 3.6.0及更高版本支持
'1_000_000, f_4240'

>>> print("The number {1:,} in hex is: {1:#x}, the number {0} in
oct is {0:o}".format(5555, 55))
The number 55 in hex is: 0x37, the number 5555 in oct is 12663
```



格式化的字符串常量 (f表达式)

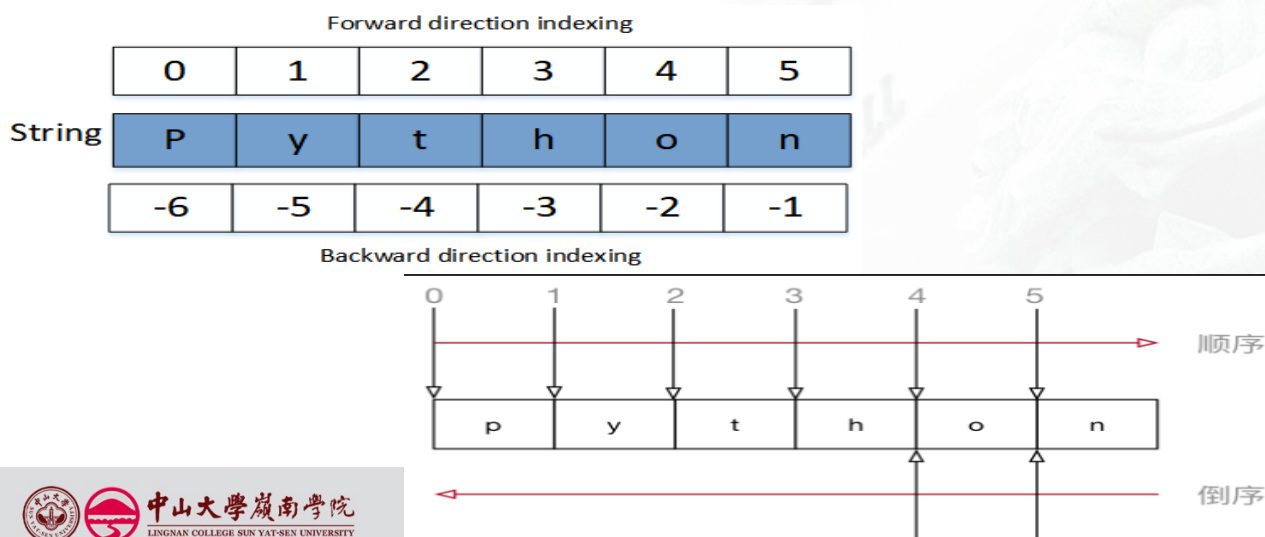
- 从Python 3.6.x开始支持一种新的字符串格式化方式，官方叫做**Formatted String Literals**，简称**f-字符串**。形式上在字符串前加字母**f**，含义与字符串对象**format()**方法类似，可以在表达式中直接用**{}**括起来的变量来进行输出

```
>>> name = 'Dong'
>>> age = 39
>>> f'My name is {name}, and I am {age} years old.'
'My name is Dong, and I am 39 years old.'
>>> width = 10
>>> precision = 4
>>> value = 11/3
>>> f'result:{value:{width}.{precision}}' # 精度：数字个数
'result:  3.667'
>>> f'result:{value:{width}.{precision}f}' # 小数位数
'result: 3.6667'
```



字符串索引与切片

- 字符串可以通过**下标索引**，引用字符，如s = "python", s[3]是'h'
- 注意Python的序列从**0**开始索引



字符串索引与切片（续）

- 字符串像列表一样，也可以使用下标切片，`a = 'bcd'`，`a[2]`是'd'，
 - `[:]` 整个字符串
 - `[start:]`，从start提取到结尾
 - `[:end]`，从开头提取到end-1，注意是取到end的前一位！！
 - `[start:end]`，从start提取到end-1
 - `[start:end:step]`，从start提取到end-1，每step个字符提取一个
 - 如：

```
spam = 'Hello world!'
print(spam[:5])      # 'Hello'
print(spam[:-1:3])   # 'Hlwl'
print(spam[::-1])    # ?
```

						[6:10]						
0	1	2	3	4	5	6	7	8	9	10	11	
M	o	n	t	y		P	y	t	h	o	n	
-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	
[-12:-7]												

字符串的运算符

- Python字符串支持加法运算符`+`，表示两个字符串连接，生成新字符串

```
>>> 'hello' + 'world'
'hello world'
```

- Python字符串支持与整数的乘法运算`*`，表示序列重复，也就是字符串内容的重复，得到新字符串

```
>>> 'abcd' * 3
'abcdabcdabcd'
```

- 成员运算符，成员（/子串）判断，关键字`in`

```
>>> "a" in "abcde"  #测试一个字符中是否存在于另一个字符串中
True
>>> 'ac' in 'abcde' #关键字in左边的字符串作为一个整体对待
False
```


适用于字符串对象的内置函数

```
>>> x = 'Hello world.'
```

```
>>> len(x), max(x), min(x)
(12, 'w', ' ')
```

```
>>> sorted(x)
[' ', '.', 'H', 'd', 'e', 'l', 'l', 'l', 'o', 'o', 'r', 'w']
```

- 内置函数`eval()`用来把任意字符串转化为Python表达式并进行求值

```
>>> eval("3+4") # 计算表达式的值, 7
```

```
>>> a = 3
```

```
>>> b = 5
```

```
>>> eval('a+b') # 要求变量a和b已存在, 8
```

```
>>> import math
```

```
>>> eval('math.sqrt(3)') # 1.7320508075688772
```

字符串的常用方法与操作

- Python字符串对象提供了大量**方法**用于字符串的切分、连接、替换和排版等操作
- **字符串对象是不可变的**，所以字符串对象提供的涉及到字符串“修改”的方法都是**返回修改后的新字符串**，并不对原始字符串做任何修改，无一例外

属性访问运算符'

- 在Python中，应用对象的方法或属性，一般需要用属性访问运算符。（也叫点运算符），格式为：
 - 对象.方法()
- 这和内置函数的用法有差异，内置函数不用属性访问运算符

```
x = 'Hello world.'
```

```
print(len(x))      # 内置函数len()
print(x.split())   # 字符串方法split()
```

查找：find()、rfind()、index()、rindex()、count()

- find()、rfind()、index()、rindex()
- find()和rfind方法分别用来查找一个字符串在另一个字符串指定范围（默认是整个字符串）中首次和最后一次出现的索引位置，如果不存在则返回-1

```
str.find(sub[, start[, end]]); str.rfind(sub[, start[, end]])
```

- index()和rindex()方法用来返回一个字符串在另一个字符串指定范围中首次和最后一次出现的索引位置，如果不存在则抛出异常

```
str.index(sub[, start[, end]]); str.rindex(sub[, start[, end]])
```

- count()方法用来返回一个字符串在当前字符串中出现的次数

```
str.count(sub[, start[, end]])
```

使用举例

```
>>> s = "apple, peach, banana, peach, pear"
>>> s.find("peach")
6
>>> s.find("peach", 7)
19
>>> s.find("peach", 7, 20)
-1
>>> s.rfind('p')
25
>>> s.index('p')
1
>>> s.index('pe')
6
```

```
>>> s.index('pear')
25
>>> s.index('ppp')
Traceback (most recent call last):
  File "<pyshell#11>", line 1, in
<module>
    s.index('ppp')
ValueError: substring not found
>>> s.count('p')
5
>>> s.count('pp')
1
>>> s.count('ppp')
0
```

分割：split()、rsplit()

- 用方法 **split()**、**rsplit()** 进行字符串分割
- ✓ **split()** 和 **rsplit()** 方法分别用来以指定字符为分隔符，把当前字符串从左往右或从右往左分隔成多个字符串，并返回包含分隔结果的列表
 - **str.split(sep=None, maxsplit=-1)**
 - **str.rsplit(sep=None, maxsplit=-1)**

```
>>> s = 'apple, peach, banana, pear'
>>> s.split(',')
['apple', 'peach', 'banana', 'pear']
>>> s = "2017-10-31"
>>> t = s.split("-")
>>> print(t)
['2017', '10', '31']
```

分隔符sep

分割：split()、rsplit()

- 对于split()和rsplit()方法，如果不指定分隔符，则字符串中的任何空白符号（空格、换行符、制表符等）都将被认为是分隔符，把连续多个空白字符看作一个分隔符

```
>>> s = 'hello world \n\n My name is Dong '
>>> s.split()
['hello', 'world', 'My', 'name', 'is', 'Dong']
>>> s = '\n\nhello world \n\n\n My name is Dong '
>>> s.split()
['hello', 'world', 'My', 'name', 'is', 'Dong']
```



连接：join()

- 字符串连接方法join()，将由字符串元素组成的可迭代对象用指定的连接符连接起来
 - str.join(iterable)

连接符

```
>>> li = ["apple", "peach", "banana", "pear"]
>>> ','.join(li)
'apple,peach,banana,pear'
>>> '!.join(li)
'apple.peach.banana.pear'
>>> '::'.join(li)
'apple::peach::banana::pear'
```



大小写转换：lower()、upper()、capitalize()、title()、swapcase()

- 方法**lower()**、**upper()** 分别将字符串的字符全部小写、大写
- 方法**capitalize()**将字符串**首字符大写**、**title()**以**首字符大写**的方式显示**每个单词**
- 方法**swapcase()**将字符串中的字符大小写互换

```
>>> s = "What is Your Name?"
>>> s.lower()      #返回小写字符串
'what is your name?'
>>> s.upper()      #返回大写字符串
'WHAT IS YOUR NAME?'
>>> s.capitalize() #字符串首字符大写
'What is your name?'
>>> s.title()      #每个单词的首字母大写
'What Is Your Name?'
>>> s.swapcase()   #大小写互换
'wHAT IS yOUR nAME?'
```



替换：replace()

- 查找替换**replace()**，类似于Word中的“全部替换”功能

■ **str.replace(old, new[, count])**

```
>>> s = "中国， 中国"
>>> print(s)
中国， 中国
>>> s2 = s.replace("中国", "中华人民共和国") #两个参数都作为一个整体
>>> print(s2)
中华人民共和国， 中华人民共和国
```



首尾截除：strip()、rstrip()、lstrip()

- 方法**strip()**、**lstrip()**、**rstrip()** 分别删除字符串**首尾**、**尾**、**首部**的指定字符（默认空格、换行等）
 - **str.strip([chars])**

```
>>> '\n\nhello world \n\n'.strip()    # 删除空白字符
'hello world'
>>> 'aaaassddf'.strip('a')             # 删除指定字符
'ssddf'
>>> 'aaaassddf'.strip('af')
'ssdd'
>>> 'aaaassddf'aaa'.rstrip('a')        # 删除字符串右端指定字符
'aaaassddf'
>>> 'aaaassddf'aaa'.lstrip('a')        # 删除字符串左端指定字符
'ssddf'aaa'
```



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



首尾截除：strip()、rstrip()、lstrip()

- 这三个方法的**参数指定的字符串并不作为一个整体对待**，而是在原字符串的两侧、右侧、左侧删除参数字符串中包含的所有字符，**一层一层地从外往里扒**

```
>>> 'aabbccddeeffg'.strip('gbaefcd')
''
```



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



首尾判断：startswith()、endswith()

- **s.startswith(t)**、**s.endswith(t)**，判断字符串是否以指定字符串开始或结束

- **str.startswith(prefix[, start[, end]])**
- **str.endswith(suffix[, start[, end]])**
- **prefix和suffix可以是元组**

```
>>> s = 'Beautiful is better than ugly.'
>>> s.startswith('Be')      #检测整个字符串
True
>>> s.startswith('Be', 5)   #指定检测范围起始位置
False
>>> s.startswith('Be', 0, 5) #指定检测范围起始和结束位置
True
>>> import os
>>> [filename for filename in os.listdir('c:\\') if filename.endswith(('.bmp', '.jpg', '.gif'))] #注意
是元组
```

字符分类：isalnum()、isalpha()、isdigit()、isdecimal()、isnumeric()、isspace()、isupper()、islower()

- 用下面的方法测试字符串的特征：**isalnum()**是否只包含**数字或字母**、而且**非空**；**isalpha()**是否只包含**字母**、而且**非空**；**isdigit()**是否只包含**数字**，而且**非空**；**isdecimal()**是否只包含**10进制数字**，而且**非空**；**isnumeric()**是否只包含**数值型字符**、而且**非空**；**isspace()**是否只包含**空格、制表符和换行**，而且**非空**；**isupper()**所有字符是否**全为大写**、而且**非空**、**islower()**是否**全为小写**、而且**非空**

```
>>> '1234abcd'.isalnum()
True
>>> '1234abcd'.isalpha()   # 全部为英文字母时返回True
False
>>> '1234abcd'.isdigit()   # 全部为数字时返回True
False
>>> 'abcd'.isalpha()
True
>>> '1234.0'.isdigit()
False
```

isdigit()、isnumeric()、isdecimal()的差异

- **isdigit()**用于检测字符串是否只包含十进制数字字符，即0到9的数字字符
- **isdecimal()**和**isdigit()**类似
- **isnumeric()**用于检测字符串是否只包含数字字符,包括十进制数字、全角数字、罗马数字等
`'一二三'.isnumeric()` # True
`"XII".isnumeric()` # True, 可用'\u216B'



填充：center()、ljust()、rjust()

- **center()**、**ljust()**、**rjust()**，返回指定宽度的新字符串，原字符串**居中**、**左对齐**或**右对齐**出现在新字符串中，如果指定宽度大于字符串长度，则使用指定的字符（默认为空格）进行填充
 - `str.center(width[, fillchar])`
 - `str.ljust(width[, fillchar])`

```
>>> 'Hello world!'.center(20)      # 居中，以空格进行填充
'   Hello world!   '
>>> 'Hello world!'.center(20, '=') # 居中，以字符=进行填充
'====Hello world!===='
>>> 'Hello world!'.ljust(20, '=')  # 居左
'Hello world!===== '
>>> 'Hello world!'.rjust(20, '=')  # 居右
'=====Hello world!'
```



总结

- 字符串定义：单引号、双引号、三重引号
- 字符串编码：Unicode、GB2312、GBK
- 转义字符：\n, \, \', \t, \f,
- 字符串格式化：
 - %格式化，现在不太常用
 - format()格式化
 - f表达式
- 字符串索引与切片
- 字符串运算符：+, *, in
- 内置函数的应用：len(), eval(), sorted(),
- 字符串常用方法与操作：
 - 查找：find(), rfind(), index(), rindex(), count()
 - 分割：split(), rsplit()
 - 连接：join()
 - 大小写转换：lower(), upper(), capitalize(), title(), swapcase()

总结

- 字符串常用方法与操作（续）：
 - 替换：replace()
 - 首尾截除：strip(), lstrip(), rstrip()
 - 首尾判断：startswith(), endswith()
 - 字符串判断：isalnum(), isalpha(), isdigit(), isdecimal(), isnumeric(), isspace(), isupper(), islower()
 - 填充：center(), ljust(), rjust()

作业

- 下面是一首诗：

He was so proud of his little girl.
It was her very first day of school.
He walked with her to school that day,
And she held his hand all the way.
They walked together quiet and sad,
A little girl and her loving dad.

- 将这6行诗定义为一个字符串（注意：换行符也是一个字符，最后一行后没有换行）
- 提取字符串开头的13个字符；
- 提取第一个'school'；
- 在前两行中每3个字符（step=3）提取一个字符；（前两行的最后不包含换行符）
- 共有多少个'girl'？
- 是不是以'Any'开头？是不是以'her loving dad.'结尾？
- 是不是只包含字母、数字而且非空？
- 将所有字母变成大写并输出



作业上传地址

- 作业截止时间为下周二晚上11:50
- 作业要求是.py文件，务必命名为：**hw01_学号.py**
- 作业务必按照模版**hw01_template.py**的格式写，模版在课件网上下载，建议直接在模版上完成作业，**务必记得修改文件名称**
- 上传地址为
<https://pan.sysu.edu.cn/link/AAFCF0C579798F48AEB3DBCC3293125E55>，文件夹名：**Python Under**，密码为**ZvAd**
- Jupyterlab下载.py文档的方法见课件网



作业（自愿作业）

- 做一个Excel表，总结字符串
 - 总结字符串的定义方法、索引与切片方法、常用操作方法等
 - 可举例
- 以后可逐渐增加列表、元组、字典等



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



AACSB
ACCREDITED



ASSOCIATION
AMBA
ACCREDITED